MLDataR - A Package for ML datasets

Installing the NHSDataR package

To install the package use the below instructions:

```
#install.packages(MLDataR)
library(MLDataR)
```

What datasets are included

The current list of data sets are:

- **Diabetes disease prediction** - supervised machine learning classification dataset to enable the prediction of diabetic patients
- **Diabetes onset prediction** - supervised machine learning regression dataset to enable prediction of the age at which a pre-diabetic will develop diabetes
- **Heart disease prediction** - supervised machine learning classification dataset to enable the prediction of heart disease using a number of key outcome features
- **Long stayers prediction** - supervised machine learning classification dataset to enable the prediction of a patient staying in hospital longer than 7 days.
- **Thyroid disease prediction** - supervised machine learning classification dataset to allow for the prediction of thyroid disease utilising historic patient records
- **Failing Care Home classification** - classification supervised machine learning dataset to predict a failing care home by selected Datix incidents
- **Counter Strike Global Offensive** - supervised machine learning regression and classification data set to predict score or match outcome.

More and more data sets are being added, and it is my mission to have more than 50 example datasets by the end of 2022.

Thyroid Disease dataset

I will first work with the Thyroid disease dataset and inspect the variables in the data:

```
glimpse(MLDataR::thyroid_disease)
#> Rows: 3,772
#> Columns: 28
#> $ ThryroidClass                <chr> "negative", "negative", "negative", "ne~
#> $ patient_age                  <dbl> 41, 23, 46, 70, 70, 18, 59, 80, 66, 68,~
#> $ patient_gender               <dbl> 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, ~
#> $ presc_thyroxine              <dbl> 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, ~
#> $ queried_why_on_thyroxine     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ presc_anthyroid_meds         <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ sick                         <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, ~
#> $ pregnant                     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ thyroid_surgery              <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ radioactive_iodine_therapyI131 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ query_hypothyroid            <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ query_hyperthyroid           <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ lithium                      <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ goitre                       <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ tumor                        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, ~
#> $ hypopituitarism              <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ psych_condition              <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ TSH_measured                 <dbl> 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, ~
#> $ TSH_reading                  <dbl> 1.30, 4.10, 0.98, 0.16, 0.72, 0.03, NA,~
#> $ T3_measured                  <dbl> 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, ~
#> $ T3_reading                   <dbl> 2.5, 2.0, NA, 1.9, 1.2, NA, NA, 0.6, 2.~
#> $ T4_measured                  <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ T4_reading                   <dbl> 125, 102, 109, 175, 61, 183, 72, 80, 12~
#> $ thyrox_util_rate_T4U_measured <dbl> 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ thyrox_util_rate_T4U_reading <dbl> 1.14, NA, 0.91, NA, 0.87, 1.30, 0.92, 0~
#> $ FTI_measured                 <dbl> 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ FTI_reading                  <dbl> 109, NA, 120, NA, 70, 141, 78, 115, 132~
#> $ ref_src                      <chr> "SVHC", "other", "other", "other", "SVI~
```

As you can see this dataset has 28 columns and 3,772 rows. The dataset is fully documented in the help file of what each one of the items means. The next task is to use this dataset to create a ML model in TidyModels.

Create TidyModels recipe to model the thyroid dataset

This will show how to create and implement the dataset in TidyModels for a supervised ML classification task.

Data preparation

The first step will be to do the data preparation steps:

```
data("thyroid_disease")
td <- thyroid_disease
# Create a factor of the class label to use in ML model
td$ThryroidClass <- as.factor(td$ThryroidClass)
# Check the structure of the data to make sure factor has been created
str(td)
#> 'data.frame':    3772 obs. of  28 variables:
#>  $ ThryroidClass                : Factor w/ 2 levels "negative","sick": 1 1 1 1 1 1 2 1 1 ...
#>  $ patient_age                  : num  41 23 46 70 70 18 59 80 66 68 ...
#>  $ patient_gender               : num  1 1 0 1 1 1 1 1 0 1 ...
#>  $ presc_thyroxine              : num  0 0 0 1 0 1 0 0 0 0 ...
#>  $ queried_why_on_thyroxine     : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ presc_anthyroid_meds         : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ sick                         : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ pregnant                     : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ thyroid_surgery              : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ radioactive_iodine_therapyI131: num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ query_hypothyroid            : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ query_hyperthyroid           : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ lithium                      : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ goitre                       : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ tumor                        : num  0 0 0 0 0 0 0 0 1 0 ...
#>  $ hypopituitarism              : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ psych_condition              : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ TSH_measured                 : num  1 1 1 1 1 1 0 1 1 1 ...
#>  $ TSH_reading                  : num  1.3 4.1 0.98 0.16 0.72 0.03 NA 2.2 0.6 2.4 ...
#>  $ T3_measured                  : num  1 1 0 1 1 0 0 1 1 1 ...
#>  $ T3_reading                   : num  2.5 2 NA 1.9 1.2 NA NA 0.6 2.2 1.6 ...
#>  $ T4_measured                  : num  1 1 1 1 1 1 1 1 1 1 ...
#>  $ T4_reading                   : num  125 102 109 175 61 183 72 80 123 83 ...
#>  $ thyrox_util_rate_T4U_measured : num  1 0 1 0 1 1 1 1 1 1 ...
#>  $ thyrox_util_rate_T4U_reading  : num  1.14 NA 0.91 NA 0.87 1.3 0.92 0.7 0.93 0.89 ...
#>  $ FTI_measured                 : num  1 0 1 0 1 1 1 1 1 1 ...
#>  $ FTI_reading                  : num  109 NA 120 NA 70 141 78 115 132 93 ...
#>  $ ref_src                      : chr  "SVHC" "other" "other" "other" ...
```

Next I will remove the missing variable, you could try another imputation method here such as MICE, however for speed of development and building vignette, I will leave this for you to look into:

```
# Remove missing values, or choose more advaced imputation option
td <- td[complete.cases(td),]
#Drop the column for referral source
td <- td %>%
   dplyr::select(-ref_src)
```

Split the data

Next I will partition the data into a training and testing split, so I can evaluate how well the model performs on the testing set:

```
#Divide the data into a training test split
set.seed(123)
split <- rsample::initial_split(td, prop=3/4)
train_data <- rsample::training(split)
test_data <- rsample::testing(split)
```

Create a recipe with preprocessing steps

After I have split the data it is time to prepare a recipe for the preprocessing steps, here I will use the recipes package:

```
td_recipe <-
   recipe(ThryroidClass ~ ., data=train_data) %>%
   step_normalize(all_predictors()) %>%
   step_zv(all_predictors())

print(td_recipe)
#> Recipe
#>
#> Inputs:
#>
#>       role #variables
#>    outcome          1
#>  predictor         26
#>
#> Operations:
#>
#> Centering and scaling for all_predictors()
#> Zero variance filter on all_predictors()
```

This recipe links the outcome variable `ThyroidClass` and then we use a normalise function to centre and scale all the numerical outcome variables and then we will remove zero variance from the data.

Getting modelling with Parsnip

We come to the modelling step of the exercise. Here I will instantiate a random forest model for the modeeling task at hand:

```
set.seed(123)
rf_mod <-
  parsnip::rand_forest() %>%
  set_engine("ranger") %>%
  set_mode("classification")
```

Create the model workflow

Tidymodels uses the concept of workflows to stitch the ML pipeline together, so I will now create the workflow and then fit the model:

```
td_wf <-
   workflow() %>%
   workflows::add_model(rf_mod) %>%
   workflows::add_recipe(td_recipe)

print(td_wf)
#> == Workflow ====================================================================
#> Preprocessor: Recipe
#> Model: rand_forest()
#>
#> -- Preprocessor ----------------------------------------------------------------
#> 2 Recipe Steps
#>
#> * step_normalize()
#> * step_zv()
#>
#> -- Model ----------------------------------------------------------------------
#> Random Forest Model Specification (classification)
#>
#> Computational engine: ranger
# Fit the workflow to our training data
set.seed(123)
td_rf_fit <-
   td_wf %>%
   fit(data = train_data)
# Extract the fitted data
td_fitted <- td_rf_fit %>%
    extract_fit_parsnip()
```

Make predictions and evaluate with ConfusionTableR

The final step, before deploying this live, would be to make predictions on the test set and then evaluate with the ConfusionTableR package:

```
# Predict the test set on the training set to see model performance
class_pred <- predict(td_rf_fit, test_data)
td_preds <- test_data %>%
    bind_cols(class_pred)
```

```r
# Convert both to factors
td_preds$.pred_class <- as.factor(td_preds$.pred_class)
td_preds$ThryroidClass <- as.factor(td_preds$ThryroidClass)

str(td_preds)
#> 'data.frame':    688 obs. of  28 variables:
#>  $ ThryroidClass                : Factor w/ 2 levels "negative","sick": 2 2 1 1 1 1 1 1 1 1 ...
#>  $ patient_age                  : num  80 81 73 78 48 81 68 27 54 74 ...
#>  $ patient_gender               : num  1 0 1 1 0 1 0 1 1 0 ...
#>  $ presc_thyroxine              : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ queried_why_on_thyroxine     : num  0 0 0 0 1 0 0 0 0 0 ...
#>  $ presc_anthyroid_meds         : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ sick                         : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ pregnant                     : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ thyroid_surgery              : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ radioactive_iodine_therapyI131: num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ query_hypothyroid            : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ query_hyperthyroid           : num  0 0 0 0 1 0 0 0 0 0 ...
#>  $ lithium                      : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ goitre                       : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ tumor                        : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ hypopituitarism              : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ psych_condition              : num  0 0 0 0 0 0 0 0 0 0 ...
#>  $ TSH_measured                 : num  1 1 1 1 1 1 1 1 1 1 ...
#>  $ TSH_reading                  : num  2.2 1.9 1.9 0.5 5.4 0.2 0.4 15 19 1 ...
#>  $ T3_measured                  : num  1 1 1 1 1 1 1 1 1 1 ...
#>  $ T3_reading                   : num  0.6 0.3 1.5 1.9 1.9 2.2 2.2 1.6 2.2 2.1 ...
#>  $ T4_measured                  : num  1 1 1 1 1 1 1 1 1 1 ...
#>  $ T4_reading                   : num  80 102 113 81 87 133 117 82 83 77 ...
#>  $ thyrox_util_rate_T4U_measured: num  1 1 1 1 1 1 1 1 1 1 ...
#>  $ thyrox_util_rate_T4U_reading : num  0.7 0.96 1.06 0.83 1 0.78 0.86 0.82 1.03 0.91 ...
#>  $ FTI_measured                 : num  1 1 1 1 1 1 1 1 1 1 ...
#>  $ FTI_reading                  : num  115 106 106 98 87 171 136 100 81 84 ...
#>  $ .pred_class                  : Factor w/ 2 levels "negative","sick": 2 2 1 1 1 1 1 1 1 1 ...

# Evaluate the data with ConfusionTableR
cm <- binary_class_cm(td_preds$.pred_class,
                      td_preds$ThryroidClass,
                      positive="sick")
#> [INFO] Building a record level confusion matrix to store in dataset
#> [INFO] Build finished and to expose record level cm use the record_level_cm list item
```

Final step is to view the Confusion Matrix and collapse down for storage in a database to model accuracy drift over time:

```r
#View Confusion matrix
cm$confusion_matrix
#> Confusion Matrix and Statistics
#>
#>           Reference
#> Prediction negative sick
#>   negative      629   11
#>   sick            3   45
#>
#>                Accuracy : 0.9797
#>                  95% CI : (0.9661, 0.9888)
#>     No Information Rate : 0.9186
#>     P-Value [Acc > NIR] : 5.472e-12
#>
#>                   Kappa : 0.8544
#>
#>  Mcnemar's Test P-Value : 0.06137
#>
#>             Sensitivity : 0.80357
#>             Specificity : 0.99525
#>          Pos Pred Value : 0.93750
#>          Neg Pred Value : 0.98281
#>              Prevalence : 0.08140
#>          Detection Rate : 0.06541
#>    Detection Prevalence : 0.06977
#>       Balanced Accuracy : 0.89941
#>
#>        'Positive' Class : sick
#>
#View record level
cm$record_level_cm
#>   Pred_negative_Ref_negative Pred_sick_Ref_negative Pred_negative_Ref_sick
#> 1                        629                      3                     11
#>   Pred_sick_Ref_sick  Accuracy     Kappa AccuracyLower AccuracyUpper
#> 1                 45 0.9796512 0.8544487     0.9660936     0.9888315
#>   AccuracyNull AccuracyPValue McnemarPValue Sensitivity Specificity
#> 1    0.9186047   5.471829e-12    0.06136883   0.8035714   0.9952532
#>   Pos.Pred.Value Neg.Pred.Value Precision    Recall        F1 Prevalence
#> 1         0.9375      0.9828125    0.9375 0.8035714 0.8653846 0.08139535
#>   Detection.Rate Detection.Prevalence Balanced.Accuracy               cm_ts
#> 1     0.06540698           0.06976744         0.8994123 2022-10-03 15:44:43
```

That is an example of how to model the Thyroid dataset, and random forest ensembles are giving us good estimates of the model performance. The Kappa level is also excellent, meaning that the model has a high likelihood of being good in practice.

Diabetes dataset

The diabetes dataset can be loaded from the package with ease also:

```r
glimpse(MLDataR::diabetes_data)
#> Rows: 520
#> Columns: 17
#> $ Age              <dbl> 40, 58, 41, 45, 60, 55, 57, 66, 67, 70, 44, 38, 35, 6~
#> $ Gender           <chr> "Male", "Male", "Male", "Male", "Male", "Male", "Male~
#> $ ExcessUrination  <chr> "No", "No", "Yes", "No", "Yes", "Yes", "Yes", "Yes", ~
#> $ Polydipsia       <chr> "Yes", "No", "No", "No", "Yes", "Yes", "Yes", "Yes", ~
#> $ WeightlossSudden <chr> "No", "No", "No", "Yes", "Yes", "No", "No", "Yes", "N~
#> $ Fatigue          <chr> "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "Yes~
#> $ Polyphagia       <chr> "No", "No", "Yes", "Yes", "Yes", "Yes", "Yes", "No", ~
#> $ GenitalThrush    <chr> "No", "No", "No", "Yes", "No", "No", "Yes", "No", "Ye~
#> $ BlurredVision    <chr> "No", "Yes", "No", "No", "Yes", "Yes", "No", "Yes", "~
#> $ Itching          <chr> "Yes", "No", "Yes", "Yes", "Yes", "Yes", "No", "Yes",~
#> $ Irritability     <chr> "No", "No", "No", "No", "Yes", "No", "No", "Yes", "Ye~
#> $ DelayHealing     <chr> "Yes", "No", "Yes", "Yes", "Yes", "Yes", "Yes", "No",~
#> $ PartialPsoriasis <chr> "No", "Yes", "No", "No", "Yes", "No", "Yes", "Yes", "~
#> $ MuscleStiffness  <chr> "Yes", "No", "Yes", "No", "Yes", "Yes", "No", "Yes", ~
#> $ Alopecia         <chr> "Yes", "Yes", "Yes", "No", "Yes", "Yes", "No", "No", ~
#> $ Obesity          <chr> "Yes", "No", "No", "No", "Yes", "Yes", "No", "No", "Y~
#> $ DiabeticClass    <chr> "Positive", "Positive", "Positive", "Positive", "Posi~
```

Has a number of variables that are common with people of diabetes, however some dummy encoding would be needed of the Yes / No variables to make this model work.

This is another example of a dataset that you could build an ML model on.

Heart disease prediction

The final dataset, for now, in the package is the heart disease dataset. To load and work with this dataset you could use the following:

```r
data(heartdisease)
# Convert diabetes data to factor'
hd <- heartdisease %>%
 mutate(HeartDisease = as.factor(HeartDisease))
is.factor(hd$HeartDisease)
#> [1] TRUE
```

Dummy encode the dataset

The ConfusionTableR package has a `dummy_encoder` function baked into the package. To code up the dummy variables you could use an approach similar to below:

```r
# Get categorical columns
hd_cat <- hd  %>%
  dplyr::select_if(is.character)
# Dummy encode the categorical variables
 cols <- c("RestingECG", "Angina", "Sex")
# Dummy encode using dummy_encoder in ConfusionTableR package
coded <- ConfusionTableR::dummy_encoder(hd_cat, cols, remove_original = TRUE)
#>
#> Attaching package: 'purrr'
#> The following object is masked from 'package:caret':
#>
#>     lift
#> Joining, by = c("Sex", "row")
#> Joining, by = c("Angina", "row", "RestingECG")
coded <- coded %>%
     select(RestingECG_ST, RestingECG_LVH, Angina=Angina_Y,
     Sex=Sex_F)
# Remove column names we have encoded from original data frame
hd_one <- hd[,!names(hd) %in% cols]
# Bind the numerical data on to the categorical data
hd_final <- bind_cols(coded, hd_one)
# Output the final encoded data frame for the ML task
glimpse(hd_final)
#> Rows: 918
#> Columns: 11
#> $ RestingECG_ST    <dbl> 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0,~
#> $ RestingECG_LVH   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
#> $ Angina           <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
#> $ Sex              <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
#> $ Age              <dbl> 40, 49, 37, 48, 54, 39, 45, 54, 37, 48, 37, 58, 39, 4~
#> $ RestingBP        <dbl> 140, 160, 130, 138, 150, 120, 130, 110, 140, 120, 130~
#> $ Cholesterol      <dbl> 289, 180, 283, 214, 195, 339, 237, 208, 207, 284, 211~
#> $ FastingBS        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
#> $ MaxHR            <dbl> 172, 156, 98, 108, 122, 170, 170, 142, 130, 120, 142,~
#> $ HeartPeakReading <dbl> 0.0, 1.0, 0.0, 1.5, 0.0, 0.0, 0.0, 0.0, 1.5, 0.0, 0.0~
#> $ HeartDisease     <fct> 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0,~
```

The data is now ready for modelling in the same fashion as we saw with the thyroid dataset.

Long stayers

This is a dataset for long stay patients and has been created off the back of real NHS data. Load in the data and the required packages:

```r
library(MLDataR)
library(dplyr)
library(ggplot2)
library(caret)
library(rsample)
library(varhandle)

data("long_stayers")
glimpse(long_stayers)
#> Rows: 768
#> Columns: 9
#> $ stranded.label           <chr> "Not Stranded", "Not Stranded", "Not Stranded~
#> $ age                      <int> 50, 31, 32, 69, 33, 75, 26, 64, 53, 63, 30, 7~
#> $ care.home.referral       <int> 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, ~
#> $ medicallysafe            <int> 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, ~
#> $ hcop                     <int> 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, ~
#> $ mental_health_care       <int> 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, ~
#> $ periods_of_previous_care <int> 1, 1, 1, 1, 1, 1, 1, 1, 5, 1, 1, 1, 1, 1, 4, ~
#> $ admit_date               <chr> "29/12/2020", "11/12/2020", "19/01/2021", "07~
#> $ frailty_index            <chr> "No index item", "No index item", "No index i~
```

Do some feature engineering on the dataset:

```r
long_stayers <- long_stayers %>%
  dplyr::mutate(stranded.label=factor(stranded.label)) %>%
  dplyr::select(everything(), -c(admit_date))

cats <- select_if(long_stayers, is.character)
cat_dummy <- varhandle::to.dummy(cats$frailty_index, "frail_ind")
#Converts the frailty index column to dummy encoding and sets a column called "frail_ind" prefix
cat_dummy <- cat_dummy %>%
  as.data.frame() %>%
  dplyr::select(-frail_ind.No_index_item) #Drop the field of interest
# Drop the frailty index from the stranded data frame and bind on our new encoding categorical
      variables
long_stayers <- long_stayers %>%
  dplyr::select(-frailty_index) %>%
  bind_cols(cat_dummy) %>% na.omit(.)
```

Then we will split and model the data. This uses the CARET package to do the modelling:

```r
split <- rsample::initial_split(long_stayers, prop = 3/4)
train <- rsample::training(split)
test <- rsample::testing(split)
```

```
set.seed(123)
glm_class_mod <- caret::train(factor(stranded.label) ~ ., data = train,
                  method = "glm")
print(glm_class_mod)
#> Generalized Linear Model
#>
#> 524 samples
#>   9 predictor
#>   2 classes: 'Not Stranded', 'Stranded'
#>
#> No pre-processing
#> Resampling: Bootstrapped (25 reps)
#> Summary of sample sizes: 524, 524, 524, 524, 524, 524, ...
#> Resampling results:
#>
#>   Accuracy   Kappa
#>   0.7691858  0.4479316
```

Next, we will make predictions on the model:

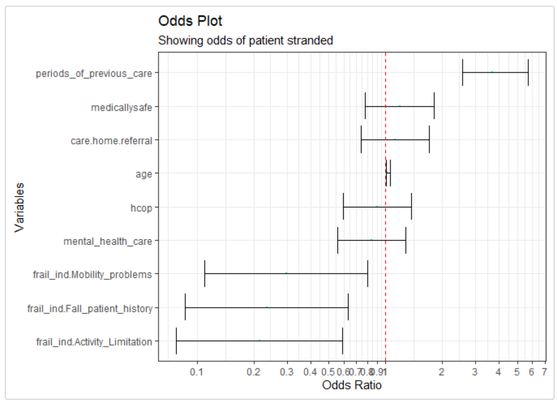
```
split <- rsample::initial_split(long_stayers, prop = 3/4)
train <- rsample::training(split)
test <- rsample::testing(split)

set.seed(123)
glm_class_mod <- caret::train(factor(stranded.label) ~ ., data = train,
                  method = "glm")
print(glm_class_mod)
#> Generalized Linear Model
#>
#> 524 samples
#>   9 predictor
#>   2 classes: 'Not Stranded', 'Stranded'
#>
#> No pre-processing
#> Resampling: Bootstrapped (25 reps)
#> Summary of sample sizes: 524, 524, 524, 524, 524, 524, ...
#> Resampling results:
#>
#>   Accuracy   Kappa
#>   0.7843472  0.4377841
```

Predicting on the test set to do the evaluation:

```
preds <- predict(glm_class_mod, newdata = test) # Predict class
pred_prob <- predict(glm_class_mod, newdata = test, type="prob") #Predict probs

# Join prediction on to actual test data frame and evaluate in confusion matrix

predicted <- data.frame(preds, pred_prob)
test <- test %>%
  bind_cols(predicted) %>%
  dplyr::rename(pred_class=preds)

glimpse(test)
#> Rows: 175
#> Columns: 13
#> $ stranded.label                <fct> Not Stranded, Not Stranded, Stranded, S~
#> $ age                           <int> 32, 64, 60, 75, 77, 67, 80, 41, 45, 46,~
#> $ care.home.referral            <int> 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, ~
#> $ medicallysafe                 <int> 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, ~
#> $ hcop                          <int> 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, ~
#> $ mental_health_care            <int> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, ~
#> $ periods_of_previous_care      <int> 1, 1, 4, 2, 4, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ frail_ind.Activity_Limitation <dbl> 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, ~
#> $ frail_ind.Fall_patient_history <dbl> 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
#> $ frail_ind.Mobility_problems   <dbl> 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, ~
#> $ pred_class                    <fct> Not Stranded, Not Stranded, Stranded, S~
#> $ Not.Stranded                  <dbl> 7.430290e-01, 8.115592e-01, 2.232220e-0~
#> $ Stranded                      <dbl> 0.2569710, 0.1884408, 0.9776778, 0.5487~
```

Finally, we can evaluate with the ConfusionTableR package and use the OddsPlotty package to visualise the odds ratios:

```
library(ConfusionTableR)
cm <- ConfusionTableR::binary_class_cm(test$stranded.label, test$pred_class, positive="Stranded")
#> [INFO] Building a record level confusion matrix to store in dataset
#> [INFO] Build finished and to expose record level cm use the record_level_cm list item
cm$record_level_cm
#>   Pred_Not.Stranded_Ref_Not.Stranded Pred_Stranded_Ref_Not.Stranded
#> 1                                108                             36
#>   Pred_Not.Stranded_Ref_Stranded Pred_Stranded_Ref_Stranded  Accuracy     Kappa
#> 1                              2                        29 0.7828571 0.4792482
#>   AccuracyLower AccuracyUpper AccuracyNull AccuracyPValue McnemarPValue
#> 1     0.7143576     0.8415195    0.8228571      0.9282341  8.636119e-08
#>   Sensitivity Specificity Pos.Pred.Value Neg.Pred.Value Precision    Recall
#> 1   0.9354839        0.75      0.4461538      0.9818182 0.4461538 0.9354839
#>         F1 Prevalence Detection.Rate Detection.Prevalence Balanced.Accuracy
#> 1 0.6041667  0.1771429      0.1657143            0.3714286         0.8427419
#>                 cm_ts
#> 1 2022-10-03 15:44:45

library(OddsPlotty)
plotty <- OddsPlotty::odds_plot(glm_class_mod$finalModel,
                                title = "Odds Plot ",
                                subtitle = "Showing odds of patient stranded",
                                point_col = "#00f2ff",
                                error_bar_colour = "black",
                                point_size = .5,
                                error_bar_width = .8,
                                h_line_color = "red")
#> Waiting for profiling to be done...
print(plotty)
#> $odds_data
#> # A tibble: 9 x 4
#>      OR  lower upper vars
#>   <dbl>  <dbl> <dbl> <chr>
#> 1 1.03  1.01   1.05  age
#> 2 1.12  0.742  1.71  care.home.referral
#> 3 1.19  0.781  1.80  medicallysafe
#> 4 0.905 0.597  1.37  hcop
#> 5 0.845 0.556  1.28  mental_health_care
#> 6 3.69  2.55   5.72  periods_of_previous_care
#> 7 0.215 0.0775 0.588 frail_ind.Activity_Limitation
#> 8 0.236 0.0864 0.635 frail_ind.Fall_patient_history
#> 9 0.298 0.110  0.802 frail_ind.Mobility_problems
#>
#> $odds_plot
```



What's on the horizon?

If you have a dataset and it is dying to be included in this package please reach out to me `@StatsGary` and I would be happy to add you to the list of collaborators.

I will be aiming to add an additional 30+ datasets to this package. All of which are at various stages of documentation, so the first version of this package will be released with the three core datasets, with more being added each additional version of the package.

Please keep watching the package GitHub, and make sure you install the latest updates of the package, when they are available.